

Question 1 (35 points)

All cameras have common characteristics including a manufacturer, model, and lens type.

Cameras provide their users with a way to take and store pictures.

Because picture storage is limited, a user cannot take pictures if they are out of storage.

Two common types of cameras are Analog, which stores pictures on film, and Digital, which stores pictures on memory cards.

Film is limited by the number of pictures that can fit on a film roll and memory cards are limited by the storage size in megabytes (MB).

Camera Type	Attributes	Behavior
All	Manufacturer Model Lens Type	Take Picture – all cameras take pictures, but each camera stores the pictures in a different way.
Analog	Pictures Remaining – the number of remaining pictures that can be taken with the current loaded film roll.	Load Film Roll – load camera with a film roll of a given size, which determines how many pictures can be taken. The number of pictures that can be taken is reset to the roll size when it is loaded. Take Picture – if the number of pictures taken thus far is less than the current roll size, the picture can be taken and the method returns true. If the number of pictures taken is equal to the current roll size, the picture cannot be taken and the method returns false. To continue taking pictures the camera must be loaded with a new roll.
Digital	Picture size in MB – the size in MB for each picture taken. MB Remaining on Card – the amount of storage in MB remaining on the current memory card.	Insert Memory Card – inserts a blank memory card of a given size in MB. The MB remaining on the card is reset to the memory card size. Take Picture – if the camera's picture size in MB is less than or equal to the remaining MB on the memory card, the picture can be taken and the method returns true.

		If the camera's picture size in MB is greater than the remaining MB on the memory card, the picture cannot be taken and method returns false. To continue taking pictures, a new memory card must be inserted.
--	--	---

Part 1 (10 points)

Lens Type has a finite number of legal values: Fisheye, Wide Angle, Standard, Telephoto, Macro.

Implement a Java Type for a Lens Type.

Part 2 (25 points)

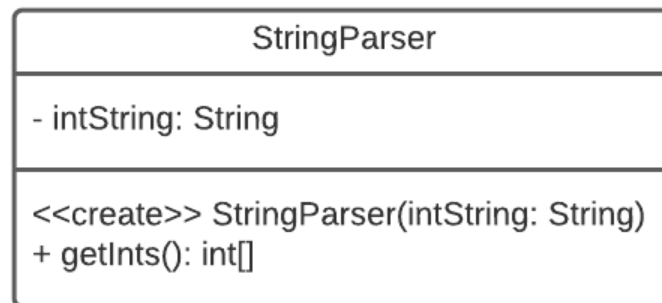
Implement each of the camera types in an appropriate class hierarchy using appropriate encapsulation, attribute/method modifier, annotations, abstraction, and inheritance techniques.

Question 2 (35 points)

In the **summation** package, you have been provided a **Summer** class. The **Summer** class relies on other classes to retrieve an array of integers from different data sources. The **Summer** class then calculates the sum of all the values in the array.

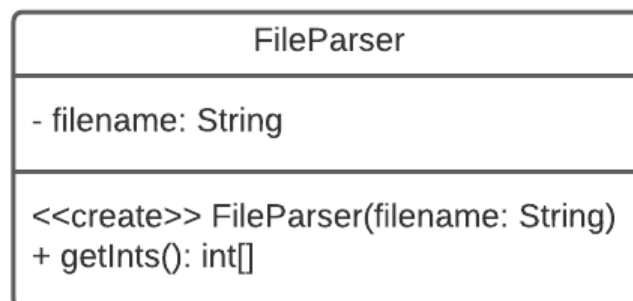
Part 1 (10 points)

Implement the **StringParser** class according to the UML diagram below. Given a comma delimited string of integers in the constructor, the **getInts** method must return an array of integers. You can assume that the class will always be instantiated with valid data, e.g. "1,2,3,4,5". Remember to commit and push your work when you complete this part!



Part 2 (10 points)

Implement the **FileParser** class according to the UML diagram below. Given a file with a single integer on each line, the **getInts** function must return an array of integers. The first line of the file is the number of integers that follow and should not be included in the array. You can assume that the filename passed to the constructor exists, and only contains integers, one per line. Remember to commit and push your work when you complete this part!



Part 3 (15 points)

Implement a **calculateSum** method in the **Summer** class. The **calculateSum** method must only accept a single parameter. Implement any necessary changes in the package to enable this, including changes to your classes written in parts 1 and 2. The **calculateSum** method should call the **getInts()** method and return the sum of all integers in the array.

Within the **Summer main** method, you may only change the **declared types** of the **StringParser** and **FileParser** variables. Remember to commit and push your work when you complete this part!

Expected Output

```
File Parser sum = 55  
String Parser sum = 10
```

Question 3 (30 points)

Before you begin, review the classes provided in the **hospital** package.

Part 1 (10 points)

Update the Nurse class so that consumers of the class cannot set the degree to anything other than LPN or RN

Part 2 (10 points)

Refactor the Nurse and Doctor classes to inherit from a common Provider class. Mark methods abstract where appropriate.

Part 3 (10 points)

Refactor the Hospital class to use a single array of Providers and a single method to retrieve the next available provider. Update the main method appropriately.